

JADAVPUR UNIVERSITY
KOLKATA, WEST BENGAL

Operating Systems Laboratory

ASSIGNMENT — II

Name Ritam Ghosh
Roll No. 002410501040
Section BCSE3.1
Semester 3 (2026–27)
Subject Operating Systems Lab (CS39002)

September 28, 2026

1. IPC Synchronization using Pipes

Question

Two processes, Process X and Process Y, interact with data x and y . Process X writes x and reads y ; Process Y writes y and reads x . Ensure proper synchronization using `fork()` and pipes.

Theory

A **pipe** is a unidirectional IPC channel with a write end and a read end. Its key property is *blocking*: a `read()` on an empty pipe suspends the calling process until data is written.

Two pipes are used for bidirectional flow:

- Pipe 1 (x flow): X writes, Y reads.
- Pipe 2 (y flow): Y writes, X reads.

Both processes follow a **Write** then **Read** pattern. If the peer has not yet written, the read blocks, providing synchronization without explicit locks.

Source Code

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/types.h>
4  #include <sys/wait.h>
5  #include <unistd.h>
6
7  int main() {
8      int pipe1[2], pipe2[2];
9      if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
10         perror("Pipe creation failed");
11         return 1;
12     }
13
14     pid_t pid = fork();
15     if (pid > 0) {
16         // Process X (Parent)
17         close(pipe1[0]);
18         close(pipe2[1]);
19
20         int x = 100;
21         printf("Process X is writing %d\n", x);
22         write(pipe1[1], &x, sizeof(x));
23
24         int y_read;
25         printf("Process X is waiting to read \n");
26         read(pipe2[0], &y_read, sizeof(y_read));
27         printf("Process X read %d\n", y_read);
28
29         close(pipe1[1]);
30         close(pipe2[0]);
31     } else {
32         // Process Y (Child)
33         close(pipe1[1]);
34         close(pipe2[0]);
35
36         int y = 200;
37         printf("Process Y is writing %d\n", y);
38         write(pipe2[1], &y, sizeof(y));
39
40         int x_read;
41         printf("Process Y is waiting to read \n");
```

```
42     read(pipe1[0], &x_read, sizeof(x_read));
43     printf("Process Y read %d\n", x_read);
44
45     close(pipe1[0]);
46     close(pipe2[1]);
47 }
48 return 0;
49 }
```

Output

Output

```
Process X is writing 100
Process X is waiting to read
Process X read 200
Process Y is writing 200
Process Y is waiting to read
Process Y read 100
```

2. Process Interleaving and Strict Synchronization

Question

- (a) Create two child processes X and Y that each run N iterations, printing their identity and iteration number with random sleep.
- (b) Enforce strict alternation: Y cannot start iteration i until X finishes iteration $i - 1$.

Theory

Process Interleaving. The OS scheduler uses time-slicing to multiplex processes. Random `usleep()` calls force context switches, producing non-deterministic output order each run.

Strict Synchronization. A pipe carries a token from X to Y. After each iteration, X writes a token; at the start of each iteration, Y reads it. The blocking property of `read()` ensures Y waits until X finishes.

Part (a) — qs2a.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/types.h>
5  #include <sys/wait.h>
6  #include <time.h>
7
8  #define ITERATIONS 10
9
10 void run_child(const char* name) {
11     srand(time(NULL) ^ getpid());
12     for (int i = 1; i <= ITERATIONS; i++) {
13         printf("Process %s - Iteration %d\n", name, i);
14         fflush(stdout);
15         usleep(rand() % 100000);
16     }
17     exit(0);
18 }
19
20 int main() {
21     pid_t pid_x = fork();
22     if (pid_x < 0) { perror("Fork failed for X"); exit(1); }
23     else if (pid_x == 0) { run_child("X"); }
24
25     pid_t pid_y = fork();
26     if (pid_y < 0) { perror("Fork failed for Y"); exit(1); }
27     else if (pid_y == 0) { run_child("Y"); }
28
29     wait(NULL);
30     wait(NULL);
31     printf("Both processes finished.\n");
32     return 0;
33 }
```

Output — Part (a)

Output

```
Process X - Iteration 1
Process Y - Iteration 1
Process Y - Iteration 2
Process Y - Iteration 3
Process X - Iteration 2
Process Y - Iteration 4
Process Y - Iteration 5
Process X - Iteration 3
Process X - Iteration 4
Process Y - Iteration 6
Process X - Iteration 5
Process Y - Iteration 7
Process X - Iteration 6
Process X - Iteration 7
Process Y - Iteration 8
Process X - Iteration 8
Process Y - Iteration 9
Process Y - Iteration 10
Process X - Iteration 9
Process X - Iteration 10
Both processes finished.
```

Part (b) — qs2b.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/types.h>
4  #include <sys/wait.h>
5  #include <time.h>
6  #include <unistd.h>
7
8  #define ITERATIONS 10
9
10 int main() {
11     int pipe_xy[2], pipe_yx[2];
12     if (pipe(pipe_xy) == -1 || pipe(pipe_yx) == -1) {
13         perror("Pipe creation failed");
14         return 1;
15     }
16
17     pid_t pid = fork();
18     if (pid < 0) { perror("Fork failed"); exit(1); }
19
20     srand(time(NULL) ^ getpid());
21     int token = 1;
22
23     if (pid > 0) {
24         // PROCESS X (Parent)
25         close(pipe_xy[0]);
26         close(pipe_yx[1]);
27
28         for (int i = 1; i <= ITERATIONS; i++) {
29             printf("Process X - Iteration %d\n", i);
30             fflush(stdout);
31             usleep(rand() % 100000);
32             write(pipe_xy[1], &token, sizeof(token));
33             if (i < ITERATIONS) {
34                 read(pipe_yx[0], &token, sizeof(token));
35             }
36         }
37         close(pipe_xy[1]);
38         close(pipe_yx[0]);
39         wait(NULL);
40         printf("Both processes finished.\n");
41     }
```

```
41 } else {  
42     // PROCESS Y (Child)  
43     close(pipe_xy[1]);  
44     close(pipe_yx[0]);  
45  
46     for (int i = 1; i <= ITERATIONS; i++) {  
47         read(pipe_xy[0], &token, sizeof(token));  
48         printf("Process Y - Iteration %d\n", i);  
49         fflush(stdout);  
50         usleep(rand() % 100000);  
51         if (i < ITERATIONS) {  
52             write(pipe_yx[1], &token, sizeof(token));  
53         }  
54     }  
55     close(pipe_xy[0]);  
56     close(pipe_yx[1]);  
57     exit(0);  
58 }  
59 return 0;  
60 }
```

Output — Part (b)

Output

```
Process X - Iteration 1  
Process Y - Iteration 1  
Process X - Iteration 2  
Process Y - Iteration 2  
Process X - Iteration 3  
Process Y - Iteration 3  
Process X - Iteration 4  
Process Y - Iteration 4  
Process X - Iteration 5  
Process Y - Iteration 5  
Process X - Iteration 6  
Process Y - Iteration 6  
Process X - Iteration 7  
Process Y - Iteration 7  
Process X - Iteration 8  
Process Y - Iteration 8  
Process X - Iteration 9  
Process Y - Iteration 9  
Process X - Iteration 10  
Process Y - Iteration 10  
Both processes finished.
```

3. IPC Mechanisms: Pipes, FIFOs, and Message Queues

Question

Implement three IPC applications:

- Weather broadcast using **Message Queues** (System V).
- Telephonic conversation using **unnamed Pipes**.
- Broadcast with listener replies using **named FIFOs**.

Theory

- **Pipes:** Unidirectional byte streams between related processes. Two pipes are needed for bidirectional communication.
- **FIFOs (Named Pipes):** Filesystem entries that allow IPC between unrelated processes via `mkfifo()`.
- **Message Queues (System V):** Kernel-managed linked lists of typed messages. The `mtype` field enables selective reception by different listeners.

Part (a) — Weather Broadcast (qs3a.c)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/ipc.h>
5  #include <sys/msg.h>
6  #include <sys/types.h>
7  #include <sys/wait.h>
8  #include <unistd.h>
9
10 #define NUM_LISTENERS 3
11 #define NUM_BROADCASTS 5
12 #define MSG_SIZE 128
13
14 struct msgbuf {
15     long mtype;
16     char mtext[MSG_SIZE];
17 };
18
19 void listener(int msgid, int listener_id) {
20     struct msgbuf msg;
21     long my_type = listener_id + 1;
22     printf(" [Listener %d] Started (PID: %d), listening on mtype=%ld\n",
23           listener_id, getpid(), my_type);
24     fflush(stdout);
25     while (1) {
26         if (msgrcv(msgid, &msg, MSG_SIZE, my_type, 0) == -1) {
27             perror(" msgrcv"); exit(1);
28         }
29         if (strcmp(msg.mtext, "END") == 0) {
30             printf(" [Listener %d] Received END. Exiting.\n", listener_id);
31             fflush(stdout); break;
32         }
33         printf(" [Listener %d] Received: \"%s\"\n", listener_id, msg.mtext);
34         fflush(stdout);
35     }
36     exit(0);
37 }
38
39 int main() {
40     printf("=====\n");
41     printf(" 3a: Weather Broadcast (Message Queue)\n");

```

```

42 printf("=====\n\n");
43 key_t key = ftok("/tmp", 'W');
44 if (key == -1) { perror("ftok"); return 1; }
45 int msgid = msgget(key, IPC_CREAT | 0666);
46 if (msgid == -1) { perror("msgget"); return 1; }
47 printf(" [Broadcaster] Message queue created (id=%d)\n\n", msgid);
48 fflush(stdout);
49
50 pid_t pids[NUM_LISTENERS];
51 for (int i = 0; i < NUM_LISTENERS; i++) {
52     pids[i] = fork();
53     if (pids[i] < 0) { perror("fork"); return 1; }
54     if (pids[i] == 0) { listener(msgid, i); exit(0); }
55 }
56 usleep(100000);
57
58 const char *weather[] = {"Sunny, 32C, Humidity 45%",
59     "Partly Cloudy, 28C, Wind 15km/h", "Thunderstorm Warning, 24C",
60     "Clear Skies, 30C, UV Index High", "Light Rain, 22C, Humidity 80%"};
61
62 struct msgbuf msg;
63 for (int b = 0; b < NUM_BROADCASTS; b++) {
64     printf("\n [Broadcaster] Sending: \"%s\"\n", weather[b]);
65     fflush(stdout);
66     for (int i = 0; i < NUM_LISTENERS; i++) {
67         msg.mtype = i + 1;
68         strncpy(msg.mtext, weather[b], MSG_SIZE - 1);
69         msg.mtext[MSG_SIZE - 1] = '\0';
70         if (msgsnd(msgid, &msg, strlen(msg.mtext) + 1, 0) == -1) {
71             perror(" msgsnd"); return 1;
72         }
73     }
74     usleep(200000);
75 }
76
77 printf("\n [Broadcaster] Sending END to all.\n");
78 fflush(stdout);
79 for (int i = 0; i < NUM_LISTENERS; i++) {
80     msg.mtype = i + 1;
81     strncpy(msg.mtext, "END", MSG_SIZE);
82     if (msgsnd(msgid, &msg, strlen(msg.mtext) + 1, 0) == -1)
83         perror(" msgsnd END");
84 }
85 for (int i = 0; i < NUM_LISTENERS; i++) waitpid(pids[i], NULL, 0);
86 if (msgctl(msgid, IPC_RMID, NULL) == -1) perror("msgctl IPC_RMID");
87 else printf("\n [Broadcaster] Queue removed. Done.\n");
88 return 0;
89 }

```

Output — Part (a)

Output

```

=====
3a: Weather Broadcast (Message Queue)
=====

[Broadcaster] Message queue created (id=65536)

[Listener 0] Started (PID: 52047), listening on mtype=1
[Listener 1] Started (PID: 52048), listening on mtype=2
[Listener 2] Started (PID: 52049), listening on mtype=3

[Broadcaster] Sending: "Sunny, 32C, Humidity 45%"
[Listener 2] Received: "Sunny, 32C, Humidity 45%"
[Listener 0] Received: "Sunny, 32C, Humidity 45%"
[Listener 1] Received: "Sunny, 32C, Humidity 45%"

[Broadcaster] Sending: "Partly Cloudy, 28C, Wind 15km/h"

```

```
[Listener 2] Received: "Partly Cloudy, 28C, Wind 15km/h"  
[Listener 0] Received: "Partly Cloudy, 28C, Wind 15km/h"  
[Listener 1] Received: "Partly Cloudy, 28C, Wind 15km/h"
```

```
[Broadcaster] Sending: "Thunderstorm Warning, 24C"  
[Listener 2] Received: "Thunderstorm Warning, 24C"  
[Listener 0] Received: "Thunderstorm Warning, 24C"  
[Listener 1] Received: "Thunderstorm Warning, 24C"
```

```
[Broadcaster] Sending: "Clear Skies, 30C, UV Index High"  
[Listener 1] Received: "Clear Skies, 30C, UV Index High"  
[Listener 0] Received: "Clear Skies, 30C, UV Index High"  
[Listener 2] Received: "Clear Skies, 30C, UV Index High"
```

```
[Broadcaster] Sending: "Light Rain, 22C, Humidity 80%"  
[Listener 0] Received: "Light Rain, 22C, Humidity 80%"  
[Listener 2] Received: "Light Rain, 22C, Humidity 80%"  
[Listener 1] Received: "Light Rain, 22C, Humidity 80%"
```

```
[Broadcaster] Sending END signal to all listeners.  
[Listener 2] Received END signal. Exiting.  
[Listener 0] Received END signal. Exiting.  
[Listener 1] Received END signal. Exiting.
```

```
[Broadcaster] Message queue removed. Done.
```

Part (b) — Telephonic Conversation (qs3b.c)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/types.h>
5  #include <sys/wait.h>
6  #include <unistd.h>
7
8  #define MSG_SIZE 256
9  #define NUM_EXCHANGES 4
10
11 int main() {
12     printf("=====\n");
13     printf(" 3b: Telephonic Conversation (Pipe)\n");
14     printf("=====\n\n");
15     fflush(stdout);
16
17     int pipe_c2r[2], pipe_r2c[2];
18     if (pipe(pipe_c2r) == -1 || pipe(pipe_r2c) == -1) {
19         perror("pipe"); return 1;
20     }
21
22     const char *caller_lines[] = {"Hello? Is this the Domino's Pizza?",
23     "I'd like to order a large pepperoni pizza.",
24     "Yes, with extra cheese please.", "Great, thanks! Goodbye."};
25     const char *receiver_lines[] = {
26     "Yes, this is Domino's! How can I help you?",
27     "Sure! Any toppings you'd like to add?",
28     "Got it! Your order will be ready in 30 minutes.",
29     "Thank you for calling! Goodbye."};
30
31     pid_t pid = fork();
32     if (pid < 0) { perror("fork"); return 1; }
33
34     char buf[MSG_SIZE];
35     if (pid > 0) {
36         // CALLER (Parent)
37         close(pipe_c2r[0]); close(pipe_r2c[1]);
38         for (int i = 0; i < NUM_EXCHANGES; i++) {
39             printf(" [Caller] : %s\n", caller_lines[i]); fflush(stdout);
40             write(pipe_c2r[1], caller_lines[i], strlen(caller_lines[i]) + 1);
41             read(pipe_r2c[0], buf, MSG_SIZE);
42             printf(" [Caller] hears: \"%s\"\n", buf); fflush(stdout);
43             usleep(300000);
44         }
45         close(pipe_c2r[1]); close(pipe_r2c[0]);
46         wait(NULL);
47         printf("\n --- Call ended ---\n");
48     } else {
49         // RECEIVER (Child)
50         close(pipe_c2r[1]); close(pipe_r2c[0]);
51         for (int i = 0; i < NUM_EXCHANGES; i++) {
52             read(pipe_c2r[0], buf, MSG_SIZE);
53             printf(" [Receiver] hears: \"%s\"\n", buf); fflush(stdout);
54             usleep(200000);
55             printf(" [Receiver] : %s\n", receiver_lines[i]); fflush(stdout);
56             write(pipe_r2c[1], receiver_lines[i], strlen(receiver_lines[i]) + 1);
57         }
58         close(pipe_c2r[0]); close(pipe_r2c[1]);
59         exit(0);
60     }
61     return 0;
62 }

```

Output — Part (b)**Output**

```
=====
3b: Telephonic Conversation (Pipe)
=====

[Caller]   : Hello? Is this the Domino's Pizza?
[Receiver] hears: "Hello? Is this the Domino's Pizza?"
[Receiver] : Yes, this is Domino's! How can I help you?
[Caller]   hears: "Yes, this is Domino's! How can I help you?"
[Caller]   : I'd like to order a large pepperoni pizza.
[Receiver] hears: "I'd like to order a large pepperoni pizza."
[Receiver] : Sure! Any toppings you'd like to add?
[Caller]   hears: "Sure! Any toppings you'd like to add?"
[Caller]   : Yes, with extra cheese please.
[Receiver] hears: "Yes, with extra cheese please."
[Receiver] : Got it! Your order will be ready in 30 minutes.
[Caller]   hears: "Got it! Your order will be ready in 30 minutes."
[Caller]   : Great, thanks! Goodbye.
[Receiver] hears: "Great, thanks! Goodbye."
[Receiver] : Thank you for calling! Goodbye.
[Caller]   hears: "Thank you for calling! Goodbye."

--- Call ended ---
```

Part (c) — Broadcast + Replies via FIFOs (qs3c.c)

```

1  #include <errno.h>
2  #include <fcntl.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6  #include <sys/stat.h>
7  #include <sys/types.h>
8  #include <sys/wait.h>
9  #include <unistd.h>
10
11 #define NUM_LISTENERS 3
12 #define NUM_BROADCASTS 4
13 #define MSG_SIZE 256
14
15 void get_bcast_path(int id, char *buf) {
16     sprintf(buf, "/tmp/qs3_bcast_%d", id);
17 }
18 void get_reply_path(int id, char *buf) {
19     sprintf(buf, "/tmp/qs3_reply_%d", id);
20 }
21
22 void listener(int id) {
23     char bcast_path[64], reply_path[64], buf[MSG_SIZE];
24     get_bcast_path(id, bcast_path);
25     get_reply_path(id, reply_path);
26     printf(" [Listener %d] Started (PID: %d)\n", id, getpid());
27     fflush(stdout);
28
29     int bcast_fd = open(bcast_path, O_RDONLY);
30     if (bcast_fd == -1) { perror(" listener open bcast"); exit(1); }
31     int reply_fd = open(reply_path, O_WRONLY);
32     if (reply_fd == -1) { perror(" listener open reply"); exit(1); }
33
34     while (1) {
35         ssize_t n = read(bcast_fd, buf, MSG_SIZE);
36         if (n <= 0) break;
37         if (strcmp(buf, "END") == 0) {
38             printf(" [Listener %d] Received END. Exiting.\n", id);
39             fflush(stdout);
40             char ack[MSG_SIZE];
41             snprintf(ack, MSG_SIZE, "Listener %d signing off.", id);
42             write(reply_fd, ack, strlen(ack) + 1);
43             break;
44         }
45         printf(" [Listener %d] Received: \"%s\"\n", id, buf);
46         fflush(stdout);
47         char reply[MSG_SIZE];
48         snprintf(reply, MSG_SIZE, "Listener %d: Acknowledged \"%s\"", id, buf);
49         write(reply_fd, reply, strlen(reply) + 1);
50     }
51     close(bcast_fd); close(reply_fd); exit(0);
52 }
53
54 int main() {
55     printf("=====\n");
56     printf(" 3c: Broadcast + Replies (Named FIFO)\n");
57     printf("=====\n\n"); fflush(stdout);
58
59     char bcast_paths[NUM_LISTENERS][64], reply_paths[NUM_LISTENERS][64];
60     for (int i = 0; i < NUM_LISTENERS; i++) {
61         get_bcast_path(i, bcast_paths[i]);
62         get_reply_path(i, reply_paths[i]);
63         unlink(bcast_paths[i]); unlink(reply_paths[i]);
64         if (mkfifo(bcast_paths[i], 0666) == -1) { perror("mkfifo"); return 1; }
65         if (mkfifo(reply_paths[i], 0666) == -1) { perror("mkfifo"); return 1; }
66     }
67     printf(" [Broadcaster] FIFOs created.\n\n"); fflush(stdout);
68
69     pid_t pids[NUM_LISTENERS];

```

```

70  for (int i = 0; i < NUM_LISTENERS; i++) {
71      pids[i] = fork();
72      if (pids[i] < 0) { perror("fork"); return 1; }
73      if (pids[i] == 0) { listener(i); exit(0); }
74  }
75
76  int bcast_fds[NUM_LISTENERS], reply_fds[NUM_LISTENERS];
77  for (int i = 0; i < NUM_LISTENERS; i++) {
78      bcast_fds[i] = open(bcast_paths[i], O_WRONLY);
79      if (bcast_fds[i] == -1) { perror("open bcast"); return 1; }
80  }
81  for (int i = 0; i < NUM_LISTENERS; i++) {
82      reply_fds[i] = open(reply_paths[i], O_RDONLY);
83      if (reply_fds[i] == -1) { perror("open reply"); return 1; }
84  }
85
86  const char *updates[] = {"Heatwave alert: 42C expected tomorrow",
87                          "Rain forecast for evening, carry umbrellas",
88                          "Air quality index: Moderate (AQI 120)",
89                          "Weekend outlook: Pleasant, 26C"};
90  char buf[MSG_SIZE];
91
92  for (int b = 0; b < NUM_BROADCASTS; b++) {
93      printf("\n [Broadcaster] Sending: \"%s\"\n", updates[b]);
94      fflush(stdout);
95      for (int i = 0; i < NUM_LISTENERS; i++)
96          write(bcast_fds[i], updates[b], strlen(updates[b]) + 1);
97      usleep(100000);
98      for (int i = 0; i < NUM_LISTENERS; i++) {
99          ssize_t n = read(reply_fds[i], buf, MSG_SIZE);
100         if (n > 0) {
101             printf(" [Broadcaster] Reply: \"%s\"\n", buf);
102             fflush(stdout);
103         }
104     }
105     usleep(200000);
106 }
107
108 printf("\n [Broadcaster] Sending END.\n"); fflush(stdout);
109 for (int i = 0; i < NUM_LISTENERS; i++) write(bcast_fds[i], "END", 4);
110 usleep(100000);
111 for (int i = 0; i < NUM_LISTENERS; i++) {
112     ssize_t n = read(reply_fds[i], buf, MSG_SIZE);
113     if (n > 0) {
114         printf(" [Broadcaster] Final: \"%s\"\n", buf);
115         fflush(stdout);
116     }
117 }
118 for (int i = 0; i < NUM_LISTENERS; i++)
119     { close(bcast_fds[i]); close(reply_fds[i]); }
120 for (int i = 0; i < NUM_LISTENERS; i++) waitpid(pids[i], NULL, 0);
121 for (int i = 0; i < NUM_LISTENERS; i++)
122     { unlink(bcast_paths[i]); unlink(reply_paths[i]); }
123 printf("\n [Broadcaster] FIFOs cleaned up. Done.\n");
124 return 0;
125 }

```

Output — Part (c)

Output

```

=====
3c: Broadcast + Replies (Named FIFO)
=====

[Broadcaster] FIFOs created.

[Listener 0] Started (PID: 52053)
[Listener 1] Started (PID: 52054)

```

```
[Listener 2] Started (PID: 52055)

[Broadcaster] Sending: "Heatwave alert: 42C expected tomorrow"
[Listener 0] Received: "Heatwave alert: 42C expected tomorrow"
[Listener 1] Received: "Heatwave alert: 42C expected tomorrow"
[Listener 2] Received: "Heatwave alert: 42C expected tomorrow"
[Broadcaster] Reply: "Listener 0: Acknowledged ..."
[Broadcaster] Reply: "Listener 1: Acknowledged ..."
[Broadcaster] Reply: "Listener 2: Acknowledged ..."

[Broadcaster] Sending: "Rain forecast for evening, carry umbrellas"
[Listener 0] Received: "Rain forecast for evening, carry umbrellas"
[Listener 1] Received: "Rain forecast for evening, carry umbrellas"
[Listener 2] Received: "Rain forecast for evening, carry umbrellas"
[Broadcaster] Reply: "Listener 0: Acknowledged ..."
[Broadcaster] Reply: "Listener 1: Acknowledged ..."
[Broadcaster] Reply: "Listener 2: Acknowledged ..."

[Broadcaster] Sending: "Air quality index: Moderate (AQI 120)"
[Listener 0] Received: "Air quality index: Moderate (AQI 120)"
[Listener 1] Received: "Air quality index: Moderate (AQI 120)"
[Listener 2] Received: "Air quality index: Moderate (AQI 120)"
[Broadcaster] Reply: "Listener 0: Acknowledged ..."
[Broadcaster] Reply: "Listener 1: Acknowledged ..."
[Broadcaster] Reply: "Listener 2: Acknowledged ..."

[Broadcaster] Sending: "Weekend outlook: Pleasant, 26C"
[Listener 0] Received: "Weekend outlook: Pleasant, 26C"
[Listener 2] Received: "Weekend outlook: Pleasant, 26C"
[Listener 1] Received: "Weekend outlook: Pleasant, 26C"
[Broadcaster] Reply: "Listener 0: Acknowledged ..."
[Broadcaster] Reply: "Listener 1: Acknowledged ..."
[Broadcaster] Reply: "Listener 2: Acknowledged ..."

[Broadcaster] Sending END to all listeners.
[Listener 0] Received END. Exiting.
[Listener 1] Received END. Exiting.
[Listener 2] Received END. Exiting.
[Broadcaster] Final: "Listener 0 signing off."
[Broadcaster] Final: "Listener 1 signing off."
[Broadcaster] Final: "Listener 2 signing off."

[Broadcaster] FIFOs cleaned up. Done.
```

4. CPU Scheduling Algorithms

Question

Build a scheduler that reads job execution profiles from a file and simulates **FCFS**, **Non-Preemptive Priority**, and **Round Robin (quantum = 16)**. Compare the average turnaround time (TAT) and waiting time (WT).

Theory

A **CPU Scheduler** selects which ready process gets the CPU next.

- **FCFS** — Non-preemptive, FIFO order. Simple but suffers from the convoy effect.
- **Priority** — Non-preemptive, lowest priority number wins. Risk of starvation for low-priority jobs.
- **Round Robin** — Preemptive, fixed time quantum. Fair but incurs context-switch overhead.

Each job has alternating CPU and I/O bursts. After a CPU burst, the job enters I/O; after I/O completes, it re-enters the ready queue. Waiting time accumulates only while in the ready queue.

Input File — jobs.txt

```
1 3 0 32 64 16 32 -1
2 1 4 64 8 32 16 8 -1
3 4 6 24 48 12 -1
4 2 8 48 32 24 64 16 -1
5 5 10 16 32 48 -1
6 1 12 64 16 32 8 16 -1
7 3 14 32 24 16 48 8 -1
8 2 16 24 16 48 32 16 -1
```

Output

Output

```
=====
CPU Scheduler Simulation
=====

Loaded 8 job profiles:
-----
Job 1 | Priority: 3 | Arrival: 0 | CPU(32)->I/O(64)->CPU(16)->I/O(32)
Job 2 | Priority: 1 | Arrival: 4 | CPU(64)->I/O(8)->CPU(32)->I/O(16)->CPU(8)
Job 3 | Priority: 4 | Arrival: 6 | CPU(24)->I/O(48)->CPU(12)
Job 4 | Priority: 2 | Arrival: 8 | CPU(48)->I/O(32)->CPU(24)->I/O(64)->CPU(16)
Job 5 | Priority: 5 | Arrival: 10 | CPU(16)->I/O(32)->CPU(48)
Job 6 | Priority: 1 | Arrival: 12 | CPU(64)->I/O(16)->CPU(32)->I/O(8)->CPU(16)
Job 7 | Priority: 3 | Arrival: 14 | CPU(32)->I/O(24)->CPU(16)->I/O(48)->CPU(8)
Job 8 | Priority: 2 | Arrival: 16 | CPU(24)->I/O(16)->CPU(48)->I/O(32)->CPU(16)
-----

===== 1. FCFS Scheduling =====

Gantt Chart:
| J1 | J2 | J3 | J4 | J5 | J6 | J7 | J8 | J1 | ...
0   32   96  120  168  184  248  280  304  320  ...

+----- FCFS (First Come First Serve) -----+
```

```

| Job   Arrival   Completion   TAT       WT       |
| 1     0         352         352        208      |
| 2     4         540         536        372      |
| 3     6         364         358        274      |
| 4     8         556         548        300      |
| 5    10         436         426        298      |
| 6    12         572         560        400      |
| 7    14         580         566        438      |
| 8    16         596         580        436      |
| Average TAT:         490.75      |
| Average WT:          340.75      |
+-----+

```

===== 2. Non-Preemptive Priority Scheduling =====

```

+----- Non-Preemptive Priority -----+
| Job   Arrival   Completion   TAT       WT       |
| 1     0         368         368        224      |
| 2     4         232         228         28      |
| 3     6         572         566        482      |
| 4     8         456         448        240      |
| 5    10         620         610        514      |
| 6    12         248         236         84      |
| 7    14         544         530        394      |
| 8    16         472         456        320      |
| Average TAT:         430.25      |
| Average WT:          285.75      |
+-----+

```

===== 3. Round Robin (quantum=16) =====

```

+----- Round Robin (q=16) -----+
| Job   Arrival   Completion   TAT       WT       |
| 1     0         336         336        184      |
| 2     4         548         544        408      |
| 3     6         316         310        226      |
| 4     8         596         588        396      |
| 5    10         428         418        322      |
| 6    12         580         568        432      |
| 7    14         468         454        326      |
| 8    16         564         548        412      |
| Average TAT:         470.75      |
| Average WT:          338.25      |
+-----+

```

Comparison

Algorithm	Avg TAT	Avg WT
FCFS	490.75	340.75
Non-Preemptive Priority	430.25	285.75
Round Robin ($q = 16$)	470.75	338.25

Priority scheduling achieves the best (lowest) average waiting time for this workload, while FCFS performs worst due to the convoy effect.

5. Deadlock Avoidance — Banker's Algorithm

Question

Implement the Banker's Algorithm to determine if a system is in a safe state and whether resource requests should be granted without leading to deadlock. Simulate until all processes finish.

Theory

The **Banker's Algorithm** avoids deadlock by checking safety before granting resources.

Key data structures:

- **Available** — Free instances per resource type.
- **Max** — Maximum demand of each process.
- **Allocation** — Currently held resources.
- **Need** — Remaining demand ($\text{Need} = \text{Max} - \text{Allocation}$).

Safety check: Find a process whose $\text{Need} \leq \text{Work}$; simulate its completion ($\text{Work} += \text{Allocation}$); repeat. If all processes finish, the state is safe.

Resource request: Verify $\text{Request} \leq \text{Need}$ and $\text{Request} \leq \text{Available}$; tentatively allocate; run safety check; grant if safe, rollback if unsafe.

Input File — input.txt

```
<#no of resources> 4 <#instances each> 2 4 5 3
<#no of processes> 3 <max needs> 1 1 1 1, 2 3 1 2, 2 2 1 3
```

Output

Output

```
=====
Banker's Algorithm Simulation
=====
Processes : 3
Resources : 4
Total      : [ 2 4 5 3 ]

+----- Current System State -----+
Available : [ 2 4 5 3 ]

      Allocation      Need
      RO R1 R2 R3      RO R1 R2 R3
P0 [ 0 0 0 0 ] [ 1 1 1 1 ]
P1 [ 0 0 0 0 ] [ 2 3 1 2 ]
P2 [ 0 0 0 0 ] [ 2 2 1 3 ]

---- Step 1 ----
P2 requests : [ 2 2 1 3 ]
-> GRANT : Safe state confirmed.
    Safe sequence: < P2 P0 P1 >
>> P2 completed. Releasing: [ 2 2 1 3 ]
    Execution order: < P2 >

+----- Current System State -----+
Available : [ 2 4 5 3 ]

      Allocation      Need
      RO R1 R2 R3      RO R1 R2 R3
```

```

P0 [ 0 0 0 0 ] [ 1 1 1 1 ]
P1 [ 0 0 0 0 ] [ 2 3 1 2 ]
P2* [ 0 0 0 0 ] [ 0 0 0 0 ]

---- Step 2 ----
P0 requests : [ 1 1 1 1 ]
-> GRANT : Safe state confirmed.
    Safe sequence: < P0 P1 P2 >
>> P0 completed. Releasing: [ 1 1 1 1 ]
    Execution order: < P2 P0 >

+----- Current System State -----+
Available : [ 2 4 5 3 ]

      Allocation      Need
      RO R1 R2 R3      RO R1 R2 R3
P0* [ 0 0 0 0 ] [ 0 0 0 0 ]
P1 [ 0 0 0 0 ] [ 2 3 1 2 ]
P2* [ 0 0 0 0 ] [ 0 0 0 0 ]

---- Step 3 ----
P1 requests : [ 2 3 1 2 ]
-> GRANT : Safe state confirmed.
    Safe sequence: < P1 P2 P0 >
>> P1 completed. Releasing: [ 2 3 1 2 ]
    Execution order: < P2 P0 P1 >

=====
All 3 processes finished successfully.
Total steps taken: 3
Final execution order: < P2 P0 P1 >
=====

```

END OF REPORT